Step 4 — A Verifier Agent That Catches the Worker's Bad Answers — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 4: the Worker can still be confidently wrong — it can misread a tool result, or answer from memory when it shouldn't. And a model re-reading its *own* answer shares the exact same blind spots, so self-checking doesn't help. This step adds a **second agent** with a different job and a different prompt: check the Worker's answer against what the tools actually returned, and flag it if it doesn't hold up. **If not:** a wrong answer would reach the user looking exactly as trustworthy as a right one.

After this step the project is complete. Files not shown here don't change.

Solution — a structured verdict, tool evidence, and one retry

Story — `schemas.py`: this is the Verifier's verdict as a Pydantic model, so it comes back checked and typed (Doc05's `with_structured_output`) — an `approved` boolean plus a reason, not prose to parse. **If not:** you'd be searching free text for "APPROVED", and a reply like "NOT APPROVED" would sneak past a naive substring check.

```
# src/schemas.py
# The shape the Verifier must answer in: VerifierVerdict (approved,
# reason).
from pydantic import BaseModel

class VerifierVerdict(BaseModel):
    approved: bool
    reason: str
```

Story — `prompts/verifier.txt`: this is a separate, strict prompt that never sees the Worker's instructions — so the Verifier *judges* the answer instead of defending it. It's filled in with the question, the proposed answer, and the tool evidence. Keeping the two prompts isolated is exactly what makes the second opinion genuinely independent. **If not:** a Verifier sharing the Worker's prompt would tend to rubber-stamp whatever the Worker said.

`src/prompts/verifier.txt`

```
You are a strict fact checker. You did not write the answer below.
Approve it only if it really answers the question AND every fact in it
matches the evidence from the tools. If no tools were used, approve it
only if the question is general knowledge that needs no live or exact data.

Question: {question}

Evidence from tools:
{evidence}

Answer to check: {answer}
```

Story — `agents/verifier.py`: this is the Verifier chain plus `collect_evidence()`, which passes along *only what the tools successfully returned* — not the Worker's failed attempts or retries. The Verifier judges the answer against that clean evidence, not the effort behind it (like a grader who sees the exam paper, not how long the student worked). **If not:** with no evidence it can only guess; handed the whole messy step log, it tends to reward effort rather than correctness.

```
# src/agents/verifier.py
# The Verifier agent: verify() checks the Worker's answer against the
# evidence from its tool calls, and returns a VerifierVerdict.
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import VerifierVerdict
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def build_verifier_chain(model):
    prompt = ChatPromptTemplate.from_template(load_prompt("verifier"))
    return prompt | model.with_structured_output(VerifierVerdict)

def collect_evidence(steps):
    evidence = ""
    for step in steps:
        if step["error"] is None:
            line = f"- {step['tool']} returned: {step['result']}\n"
            evidence = evidence + line
    if evidence == "":
        return "(no tools were used)"
    return evidence

def verify(verifier_chain, question, result):
    logger = get_logger(__name__)
    verdict = verifier_chain.invoke({
        "question": question,
```

```
        "answer": result.final_answer,
        "evidence": collect_evidence(result.steps),
    })
    logger.info("verifier: approved=%s (%s)", verdict.approved, verdict.reason)
    return verdict
```

Story — `services/research_service.py` : this holds the whole handoff in one function: the Worker answers, the Verifier checks, and on a rejection the Worker gets **one** retry with the Verifier's reason added to its task — and that retry's answer is judged against the *original* question. Keeping this flow in the service layer (not `main.py`) means it can be reused and tested on its own. (Also create an empty `services/__init__.py`.) **If not:** `main.py` would mix input handling with the agent flow, and a rejected answer would get no chance to be fixed.

```
# src/services/research_service.py
# The flow between the agents: answer_task() runs the Worker, then the
# Verifier, and on a rejection reruns the Worker once with the reason.

from src.agents.verifier import verify
from src.agents.worker import run_worker
from src.utils.logger import get_logger

def answer_task(model, verifier_chain, task, max_iterations):
    logger = get_logger(__name__)
    result = run_worker(model, task, max_iterations)
    verdict = verify(verifier_chain, task, result)

    if not verdict.approved:
        logger.warning("Verifier rejected the answer — retrying once")
        retry_task = (
            task
            + "\n\nA reviewer rejected your last answer because: "
            + verdict.reason
            + "\nCheck the facts with your tools and answer again."
        )
        result = run_worker(model, retry_task, max_iterations)
        verdict = verify(verifier_chain, task, result)

    return result, verdict
```

Changed from Step 3: `main.py` builds the Verifier chain once at startup and calls `answer_task()` instead of `run_worker()`. Every answer is now labelled verified or NOT verified. The error handling is the same.

```
# main.py
# The file you run: python main.py
```

```
# Runs each task through answer_task() — Worker, then Verifier — and
# labels every answer verified or NOT verified.
import sys

import openai

from src.agents.verifier import build_verifier_chain
from src.config import load_config
from src.exceptions import MaxIterationsExceeded, MissingConfigError
from src.models.llm import get_model
from src.services.research_service import answer_task
from src.utils.logger import get_logger

def print_steps(steps):
    for step in steps:
        if step["error"] is None:
            outcome = step["result"]
        else:
            outcome = "Error: " + step["error"]
        print(f"  step {step['step']}: {step['tool']}({step['arguments']})")
        print(f"          -> {outcome}")

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    model = get_model(config)
    # new in Step 4: the Verifier chain is built once, at startup
    verifier_chain = build_verifier_chain(model)
    print("ResearchHand — ask a question, or /quit to exit.")

    while True:
        task = input("You: ").strip()
        if task == "/quit":
            break
        if task == "":
            continue

        try:
            # changed in Step 4: Worker + Verifier, instead of Worker alone
            result, verdict = answer_task(
                model, verifier_chain, task, 5
            )
```

```
        except MaxIterationsExceeded as e:
            print(f"Gave up: {e}")
            print_steps(e.steps)
            continue
        except openai.AuthenticationError:
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        except openai.RateLimitError:
            logger.warning("Rate limited after the client's retries.")
            print("We're busy right now — wait a moment and try again.")
            continue

        print_steps(result.steps)
        # new in Step 4: the label depends on the Verifier's verdict
        if verdict.approved:
            print(f"Answer (verified): {result.final_answer}")
        else:
            print(f"Answer (NOT verified — {verdict.reason}):")
            print(f"  {result.final_answer}")

if __name__ == "__main__":
    main()
```

Changed from Step 3: `test_agents.py` adds cases 6 and 7 for `collect_evidence()` — a failed step is left out of the evidence, and no steps gives `(no tools were used)`.

```
# tests/unit/test_agents.py
# Checks the Worker's and Verifier's helper functions with hand-made
# tool calls and steps. Each case prints OK or WRONG.
# No API calls, no API key needed.
# Run from the repo root: python -m tests.unit.test_agents
# new in Step 4
from src.agents.verifier import collect_evidence
from src.agents.worker import ALL_TOOLS, find_tool, run_tool

def test_find_known_tool():
    result = "OK"
    if find_tool(ALL_TOOLS, "celsius_to_fahrenheit") is None:
        result = "WRONG"
    print(f"case 1 (known tool found)          -> {result}")

def test_find_unknown_tool():
    result = "OK"
    if find_tool(ALL_TOOLS, "send_email") is not None:
        result = "WRONG"
```

```
    print(f"case 2 (unknown tool gives None)   -> {result}")

def test_run_good_call():
    call = {"name": "celsius_to_fahrenheit", "args": {"celsius": 0}}
    output, error = run_tool(call, ALL_TOOLS)
    result = "OK"
    if output != "32.0" or error is not None:
        result = "WRONG"
    print(f"case 3 (good call gives 32.0)      -> {result}")

def test_run_unknown_tool():
    call = {"name": "send_email", "args": {}}
    output, error = run_tool(call, ALL_TOOLS)
    result = "OK"
    if output is not None or error is None:
        result = "WRONG"
    print(f"case 4 (unknown tool gives error)  -> {result}")

def test_run_bad_arguments():
    call = {"name": "celsius_to_fahrenheit", "args": {"celsius": "x"}}
    output, error = run_tool(call, ALL_TOOLS)
    result = "OK"
    if output is not None or error is None:
        result = "WRONG"
    print(f"case 5 (bad arguments give error)  -> {result}")

# new in Step 4
def test_evidence_skips_failed_steps():
    steps = [
        {"tool": "search_docs", "result": None, "error": "index down"},
        {"tool": "search_docs", "result": "Refunds within 30 days.",
         "error": None},
    ]
    evidence = collect_evidence(steps)
    result = "OK"
    if "index down" in evidence or "30 days" not in evidence:
        result = "WRONG"
    print(f"case 6 (failed step left out)      -> {result}")

# new in Step 4
def test_evidence_with_no_steps():
    result = "OK"
    if collect_evidence([]) != "(no tools were used)":
        result = "WRONG"
    print(f"case 7 (no steps)                  -> {result}")

if __name__ == "__main__":
```

```
    test_find_known_tool()
    test_find_unknown_tool()
    test_run_good_call()
    test_run_unknown_tool()
    test_run_bad_arguments()
    # new in Step 4
    test_evidence_skips_failed_steps()
    test_evidence_with_no_steps()
```

Changed from Step 3: the integration test adds cases 7 and 8. Case 7 matters most: a Verifier that has never rejected anything hasn't been tested.

Story — cases 7 and 8: these are the Verifier's own tests, built from hand-made evidence so no model is needed: an `AgentResult` whose tools said 64.4°F but whose answer claims 90°F must be **rejected**, and the same evidence (`lahore_steps()`) with the correct answer must be **approved**. Each is its own function, so a failure points straight at the gap. **If not:** a Verifier that just approves everything would look exactly like one that works.

```
# tests/integration/test_live_agents.py
# Checks the app against the real OpenAI API (and Open-Meteo) — each step
# adds cases. Each case prints OK or WRONG.
# Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents

# new in Step 4
from src.agents.verifier import build_verifier_chain, verify
from src.agents.worker import AgentResult, run_worker
from src.chains.direct_answer import build_direct_chain
from src.config import load_config
from src.exceptions import MaxIterationsExceeded
from src.models.llm import get_model
from src.tools.docs_search import search_state

def test_direct_answer(model):
    chain = build_direct_chain(model)
    answer = chain.invoke({"question": "What is the capital of France?"})
    result = "OK"
    if "Paris" not in answer:
        result = "WRONG"
    print(f"case 1 (direct answer)  -> {result}: {answer}")

def tools_used(outcome):
    names = []
    for step in outcome.steps:
        names.append(step["tool"])
    return names
```

```
def test_conversion_uses_tool(model):
    task = "What is 25 degrees Celsius in Fahrenheit?"
    used = tools_used(run_worker(model, task, 5))
    result = "OK"
    if "celsius_to_fahrenheit" not in used:
        result = "WRONG"
    print(f"case 2 (conversion)     -> {result}: {used}")

def test_no_tool_needed(model):
    outcome = run_worker(model, "What is the capital of France?", 5)
    result = "OK"
    if len(outcome.steps) != 0:
        result = "WRONG"
    print(f"case 3 (no tool needed) -> {result}: {outcome.final_answer}")

def test_step_limit(model):
    task = "What is 25 degrees Celsius in Fahrenheit?"
    result = "WRONG"
    try:
        run_worker(model, task, 1)
    except MaxIterationsExceeded:
        result = "OK"
    print(f"case 4 (step limit)     -> {result}")

def test_weather_uses_tool(model):
    task = "What's the weather in Lahore right now?"
    used = tools_used(run_worker(model, task, 5))
    result = "OK"
    if "get_weather" not in used:
        result = "WRONG"
    print(f"case 5 (weather task)   -> {result}: {used}")

def test_tool_failure_recovery(model):
    search_state.call_count = 0
    outcome = run_worker(model, "What is your refund policy?", 5)
    failed = 0
    succeeded = 0
    for step in outcome.steps:
        if step["error"] is None:
            succeeded = succeeded + 1
        else:
            failed = failed + 1
    result = "OK"
    if failed == 0 or succeeded == 0:
        result = "WRONG"
    print(f"case 6 (recovery)       -> {result}: {failed} failed, "
```

```
          f"{succeeded} succeeded")

# new in Step 4
def lahore_steps():
    return [
        {"tool": "get_weather", "result": "Currently 18°C in Lahore",
         "error": None},
        {"tool": "celsius_to_fahrenheit", "result": "64.4", "error": None},
    ]

# new in Step 4
def test_verifier_rejects_wrong_answer(verifier_chain):
    wrong = AgentResult(
        final_answer="It is 90°F in Lahore.", steps=lahore_steps()
    )
    verdict = verify(verifier_chain, "Weather in Lahore in °F?", wrong)
    result = "OK"
    if verdict.approved:
        result = "WRONG"
    print(f"case 7 (wrong answer)   -> {result}: {verdict.reason}")

# new in Step 4
def test_verifier_approves_right_answer(verifier_chain):
    right = AgentResult(
        final_answer="It is 64.4°F in Lahore.", steps=lahore_steps()
    )
    verdict = verify(verifier_chain, "Weather in Lahore in °F?", right)
    result = "OK"
    if not verdict.approved:
        result = "WRONG"
    print(f"case 8 (right answer)   -> {result}")

if __name__ == "__main__":
    config = load_config()
    model = get_model(config)
    # new in Step 4
    verifier_chain = build_verifier_chain(model)
    test_direct_answer(model)
    test_conversion_uses_tool(model)
    test_no_tool_needed(model)
    test_step_limit(model)
    test_weather_uses_tool(model)
    test_tool_failure_recovery(model)
    # new in Step 4
    test_verifier_rejects_wrong_answer(verifier_chain)
    test_verifier_approves_right_answer(verifier_chain)
```

Story — `data/sample/tasks.json` + `scripts/compare_with_tools.py`: this asks each sample task two ways — Step 1's from-memory chain, and the full Worker + Verifier pipeline — and prints both answers side by side. It's meant for *you* to read and compare, which is why it's a script, not a pass/fail test. **If not:** the project's whole claim — "verified answers, not guesses" — would never actually be shown in one place.

`data/sample/tasks.json`

```
{
  "tasks": [
    "What's the weather in Lahore right now, in Fahrenheit?",
    "What is your refund policy?",
    "What is the capital of France?"
  ]
}
```

```
# scripts/compare_with_tools.py
# Asks every sample task two ways: from memory (Step 1's chain) and with
# the Worker + Verifier. Prints both, side by side, for a person to judge.
# Run from the repo root: python -m scripts.compare_with_tools
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
import json

from src.agents.verifier import build_verifier_chain
from src.chains.direct_answer import build_direct_chain
from src.config import load_config
from src.models.llm import get_model
from src.services.research_service import answer_task

def main():
    config = load_config()
    model = get_model(config)
    direct_chain = build_direct_chain(model)
    verifier_chain = build_verifier_chain(model)
    with open("data/sample/tasks.json") as samples_file:
        samples = json.loads(samples_file.read())

    for task in samples["tasks"]:
        print(f"\nTask: {task}")
        guess = direct_chain.invoke({"question": task})
        print(f"  from memory: {guess}")
        result, verdict = answer_task(model, verifier_chain, task, 5)
        print(f"  with tools:  {result.final_answer}")
        print(f"  verified:    {verdict.approved} ({verdict.reason})")
```

```
if __name__ == "__main__":
    main()
```

Run everything:

```
python -m tests.unit.test_config
python -m tests.unit.test_tools
python -m tests.unit.test_agents
python -m tests.integration.test_live_agents
python -m scripts.compare_with_tools
python main.py
```

Expected output of the new unit and integration lines (the reason's wording varies; wrapped to fit the page):

```
case 6 (failed step left out)      -> OK
case 7 (no steps)                  -> OK
```

```
case 7 (wrong answer)   -> OK: The answer says 90°F, but the tools
                           returned 64.4°F.
case 8 (right answer)   -> OK
```

Expected app session (the log lines are left out; wording varies):

```
You: What's the weather in Lahore in Fahrenheit?
  step 0: get_weather({'city': 'Lahore'})
          -> Currently 18.0°C in Lahore
  step 1: celsius_to_fahrenheit({'celsius': 18.0})
          -> 64.4
Answer (verified): It's currently 64.4°F in Lahore.
```